udp UNIVERSIDAD DIEGO PORTALES

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Informe Tarea 2 — Sistemas Operativos
Torneo de Komikan Concurrente

Estudiantes: Francisco Grandon, Alexis Godoy **Profesor:** Víctor Reyes

1. Diseño del esquema de sincronización

El sistema utiliza hilos POSIX (`pthread`) y semáforos `sem_t`. Como todos los componentes se ejecutan dentro de un mismo proceso, no es necesario utilizar memoria compartida System V, lo que simplifica el manejo de los recursos. La estructura de hilos se organiza de la siguiente manera:

- **Torneo:** N hilos cliente que llegan de forma escalonada, el hilo principal como *matchmaker*, un hilo para recibir señales y un hilo por cada mesa activa.
- **Mesa:** un hilo Juez y hasta 13 hilos de fichas (1 Puma y 12 Perros), que comparten el mismo tablero.

Tablero y exclusión mutua. El tablero se representa mediante un grafo estático de 28 nodos (25 de la cuadrícula 5×5 y los tres restantes correspondientes a `TRONO_TOP/LEFT/RIGHT`). Las tablas de adyacencia y de saltos se construyen una sola vez con `pthread_once` y luego se mantienen sin modificaciones. El estado del tablero (`cell[]`, posiciones y perros capturados) está protegido por un único mutex por mesa (`Board.lock`). Cada movimiento (`board_move_puma`, `board_move_dog`) valida y actualiza el tablero dentro de la misma sección crítica. Así, si varias fichas intentan ocupar una misma casilla, solo la primera en tomar el mutex puede hacerlo; las demás reciben `MOVE_ERR_OCCUPIED`. De esta manera, se evita que el tablero cambie entre la validación y la ejecución del movimiento. Las capturas también se actualizan dentro de esta sección crítica.

Primitivas utilizadas.

Primitiva	Propósito
`Board.lock` (mutex)	Protege el estado del tablero y evita modificaciones simultáneas.
`sem_t g_mesas_libres`	Semáforo inicializado en M que limita la cantidad de mesas activas. Antes de lanzar una mesa, el *matchmaker* hace `sem_wait`; si no hay mesas libres, espera dormido (espera pasiva). Cada mesa que termina hace `sem_post`.
`g_q_mutex` + `g_q_cond`	Protegen la cola de jugadores. El *matchmaker* espera con `cond_wait` hasta que haya dos jugadores disponibles.
`Mesa.mu` + `q_cv`	Coordinan la cola de eventos entre las fichas y el Juez, quien procesa los eventos, imprime el tablero y revisa si hay un ganador.
`sleep_cv` + `done` (atómico)	Permiten realizar retardos aleatorios de 50–300 ms mediante `cond_timedwait`, interrumpibles cuando termina la partida o se solicita el cierre.
`g_apagado` (atómico)	Indica si el programa está en proceso de apagado y es consultada por el Juez y el *matchmaker*.

Reproducibilidad. Cada ficha tiene su propio generador de números pseudoaleatorios (xorshift), cuya semilla depende de la semilla de `config.txt`, el id de la mesa y el slot de la ficha. Así, no se comparte un generador global y no se necesita un mutex para generar números aleatorios. El archivo permite configurar `SEED`, `DELAY_MIN/MAX`, `PUMA_INIT` y los eventos de cada caso de prueba.

Resiliencia. Ante un abandono, el evento `DISCONNECT` hace que la ficha ejecute `abandon()`, registre el evento y finalice la partida a favor del oponente. La función `finish()` notifica a los hilos en espera para que puedan terminar correctamente. Como `Board.lock` solo se utiliza dentro de las funciones `board_*`, se reduce el riesgo de que una ficha termine mientras mantiene el tablero bloqueado. Al cerrar la mesa, se hace `join` de los hilos y se destruyen los mutex y las variables de condición.

Para manejar `SIGINT`, la señal se bloquea antes de crear los hilos y un hilo dedicado la recibe mediante `sigwait`. Al recibirla, activa `g_apagado`, notifica a las colas y ejecuta `sem_post` para despertar al *matchmaker*, que puede estar dormido en la cola o en el semáforo. Se rechazan nuevos clientes, los Jueces detectan el apagado y las partidas se cierran. Finalmente, se espera la terminación de los hilos (`join`) y se liberan los recursos con `sem_destroy` y `free`.

2. Análisis de riesgos de concurrencia

Deadlock. Para evitar interbloqueos, se diseñó el sistema de manera que ningún hilo mantenga dos mutex tomados al mismo tiempo. `Board.lock` no se adquiere mientras se mantiene `Mesa.mu`, ni viceversa. Las fichas obtienen primero el resultado del movimiento y después encolan el evento; el Juez consulta el tablero fuera de `Mesa.mu`. Además, las secciones críticas del tablero son breves y no incluyen operaciones de entrada o salida ni esperas bloqueantes. La impresión queda a cargo del Juez y tampoco se llama a `sem_wait` mientras se mantiene `g_q_mutex`.

Livelock. Este problema podría ocurrir si las fichas repitieran intentos de movimiento sin lograr avanzar. Para reducirlo, cada ficha espera un tiempo aleatorio entre intentos y, si encuentra una casilla ocupada, prueba con otro vecino. Además, los candidatos se mezclan en cada turno. Estos retardos ayudan a evitar que varias fichas repitan los mismos intentos de forma sincronizada.

Starvation. Aunque `pthread_mutex` no garantiza un orden FIFO, las secciones críticas son breves en comparación con los retardos de 50–300 ms, por lo que se espera una baja contención. Además, las partidas en modo libre tienen un límite de intentos para evitar que se prolonguen indefinidamente. En el *matchmaker*, los jugadores se atienden según su orden de llegada, evitando que las primeras parejas queden postergadas. Los semáforos POSIX tampoco garantizan un orden FIFO estricto, pero esto no afecta a nuestro diseño: el único hilo que espera en `g_mesas_libres` es el *matchmaker*, por lo que no hay varios hilos compitiendo por el mismo semáforo.

Otros riesgos tratados. Para reducir el riesgo de *lost wakeups*, los cambios de estado (`done`, `stop`, `g_apagado`) se coordinan con los mutex correspondientes y se notifican mediante `broadcast`. Las esperas temporizadas del Juez también permiten revisar periódicamente el estado. Por otra parte, `atomic_exchange(&done)` evita que varios hilos establezcan resultados distintos al finalizar una partida: solo el primero que cambia el estado puede definir el resultado.

3. Evaluación

Lock grueso vs. fino. Se eligió un único mutex por tablero en lugar de uno por casilla. Un salto involucra tres posiciones (origen, perro capturado y destino), por lo que usar mutex separados exigiría definir un orden global de adquisición para evitar deadlocks. En cambio, con un solo mutex, la validación y actualización del movimiento se realizan de forma conjunta. Como las secciones críticas son breves y las fichas actúan cada 50–300 ms, se espera que la contención sea baja, manteniendo una implementación más simple y segura. El costo es que dos movimientos en casillas distintas del mismo tablero no se ejecutan en paralelo, aunque en la práctica esto casi no se nota.

Concurrencia entre mesas. Cada mesa tiene su propio tablero y mutex, por lo que las partidas avanzan de manera independiente. Los recursos compartidos principales son el semáforo de mesas libres y la cola del *matchmaker*, que se utilizan para coordinar la disponibilidad de mesas y la formación de parejas.

Hilos vs. procesos. Se eligieron hilos porque las fichas y el Juez necesitan acceder al mismo tablero. Esto permite compartir directamente las estructuras de datos y sincronizar el acceso mediante mutex y variables de condición, sin recurrir a memoria compartida entre procesos. Además, el cierre se simplifica mediante `join` y la liberación de los recursos utilizados.

Juez desacoplado. La impresión del tablero se deja a cargo del Juez para evitar mantener bloqueado el mutex durante las operaciones de salida. Mediante una cola de eventos, las fichas pueden registrar sus movimientos sin encargarse directamente de imprimir el tablero. Como consecuencia, el registro mostrado puede tener un pequeño retraso respecto del estado real de la partida.

Pruebas. La implementación se probó con los 12 casos de `test_cases.txt` y con las pruebas del tablero (`make test`, que incluye 4 piezas disputando la misma casilla durante 2000 rondas). Se usó `valgrind -leak-check=full` para revisar fugas de memoria, con resultado limpio en los 12 casos y también al enviar `SIGINT` durante un torneo de 20 jugadores y 2 mesas.